Contents

1 Perspective rectification **1**
- 1.1 perspective transform . . . 1
- 1.2 affine transform . . . 2
- 1.3 explanation . . . 2

2 Gausssian and Laplacian pyramid **3**
- 2.1 gabor kernels . . . 3
  - 2.1.1 0 degrees: . . . 3
  - 2.1.2 45 degrees: . . . 4
  - 2.1.3 90 degrees: . . . 5
  - 2.1.4 135 degrees: . . . 6
  - 2.1.5 result . . . 7
- 2.2 playing with parameters . . . 7
  - 2.2.1 lambda=15 . . . 8
  - 2.2.2 lambda=12 . . . 8
  - 2.2.3 lambda=8 . . . 9
  - 2.2.4 lambda=5 . . . 9

3 canny edge detection **10**

#+OPTIONS toc:nil

1 Perspective rectification

1.1 perspective transform

```
Point2f src[4] = {
    {647, 571},
    {2515, 565},
    {3019, 3401},
    {132, 3396}
};

Point2f dst[4] = {
    {0, 0},
    {210, 0},
    {210, 297},
    {0, 297}
};

warpPerspective(image, image2, getPerspectiveTransform(src, dst),
↪  Size(210,297));
```

1.2 affine transform

```
Point2f affine_src[3] = {
    {647, 571},
    {2515, 565},
    {132, 3396}
};

Point2f affine_dst[3] = {
    {0,0},
    {(float)image.cols, 0},
    {0, (float)image.rows}
};

Mat affine_matrix = getAffineTransform(affine_src, affine_dst);
warpAffine(image, image2, affine_matrix, image.size());

//manual application of affine transformation matrix to point
Point2f original_point(3019, 3401);

Point2f transformed_point;

transformed_point.x =
    affine_matrix.at<double>(0, 0) * original_point.x +
    affine_matrix.at<double>(0, 1) * original_point.y +
    affine_matrix.at<double>(0, 2);

transformed_point.y =
    affine_matrix.at<double>(1, 0) * original_point.x +
    affine_matrix.at<double>(1, 1) * original_point.y +
    affine_matrix.at<double>(1, 2);
```

The fourth corner ends up at (4752, 4100) instead of around (3019, 3401), distance 1868.66

1.3 explanation

An affine transformation preserves lines and parallelism. If the original image has non-parallel lines (like the vertical lines on our sheet of paper) they will not be parallel after the transformation. In comparison, te

2 Gausssian and Laplacian pyramid

2.1 gabor kernels

```
const double sigma = 4.0, lambda = 10.0, gamma = 0.5, psi = 0;
const int angles[4] = {0, 45, 90, 135};

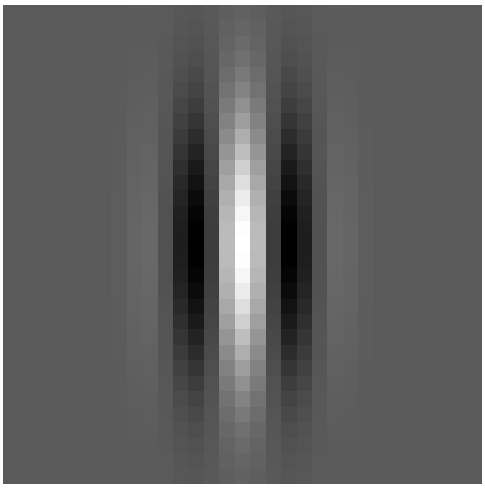
for (int i = 0; i < 4; ++i) {
  double theta = angles[i] * CV_PI / 180.0;

  // compute kernel
  Mat kernel = getGaborKernel(Size(31, 31), sigma, theta, lambda, gamma,
  ↪  psi, CV_32F);

  // filter image
  Mat resp;
  filter2D(img, resp, CV_32F, kernel);

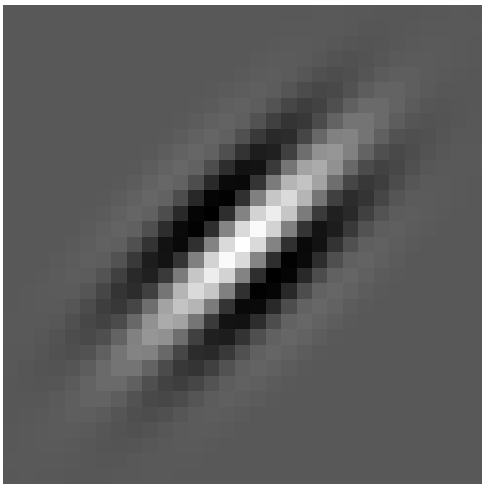
  // combine results by taking the max value of each pixel
  max(combined, resp, combined);
}
```

2.1.1 0 degrees:



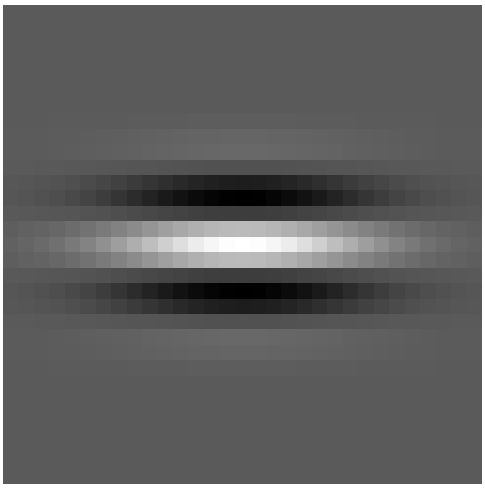


2.1.2 45 degrees:



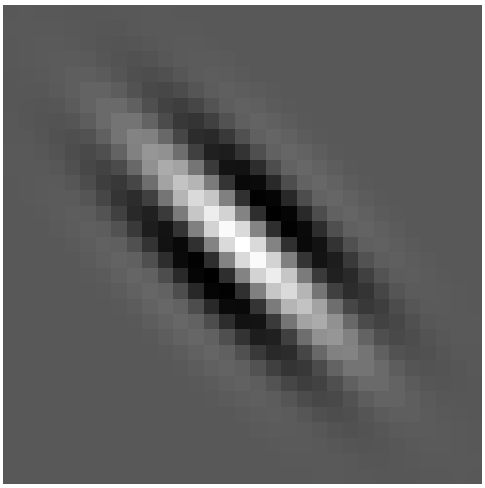


2.1.3 90 degrees:





2.1.4 135 degrees:





2.1.5 result



2.2 playing with parameters

i use

```
const int ksize = 6*sigma+1
```

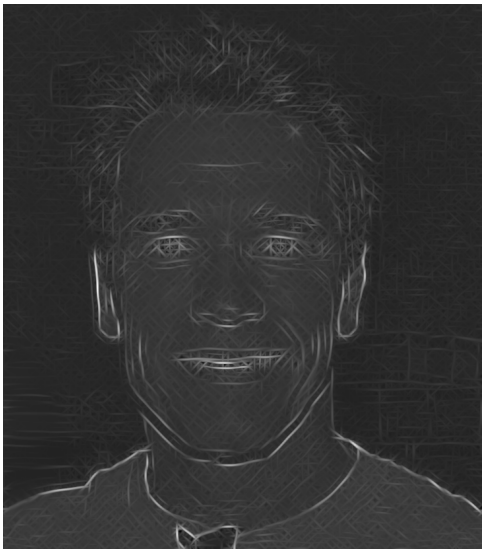
2.2.1 lambda=15



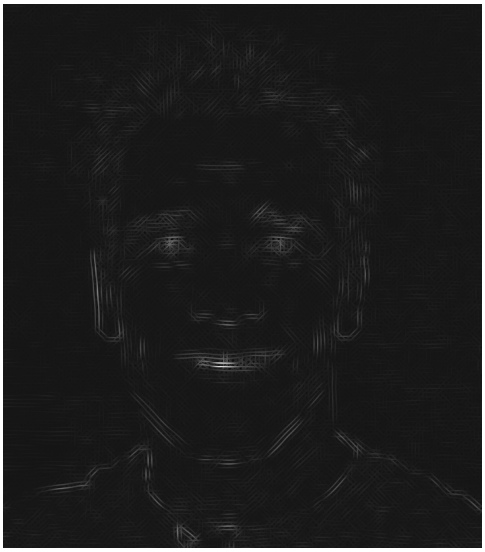
2.2.2 lambda=12



2.2.3 lambda=8



2.2.4 lambda=5



For faces a lambda of around 8 seems best.

3 canny edge detection

```
double otsuThreshold = threshold(img, otsuImage, 0, 255, THRESH_BINARY |
↪  THRESH_OTSU);
double lowerThreshold = 0.5 * otsuThreshold;
double upperThreshold = otsuThreshold;
Mat edges;
Canny(img, edges, lowerThreshold, upperThreshold);
```

Unfortunately, there was no builtin for generating multiple thresholds using otsus method. We decided to use a fixed 2:1 ratio based on otsu istead of implementing our own threshold function.

